

Unit 2 · Topic Notes

React Component Architecture, Routing & State Management

Course: Full Stack Application Development with MERN

Learning Objectives

1. Structure a UI as a tree of reusable, composable components
 2. Pass data down with props and lift state up when siblings need to share it
 3. Avoid prop drilling using the Context API
 4. Set up client-side routing with React Router (routes, links, dynamic params, nested layouts)
 5. Plan a page structure the same way real MERN apps are organized
-

1. From HTML to Components

A React component is a JavaScript function that returns HTML-like syntax (JSX) — think of it as a custom, reusable HTML tag you define yourself.

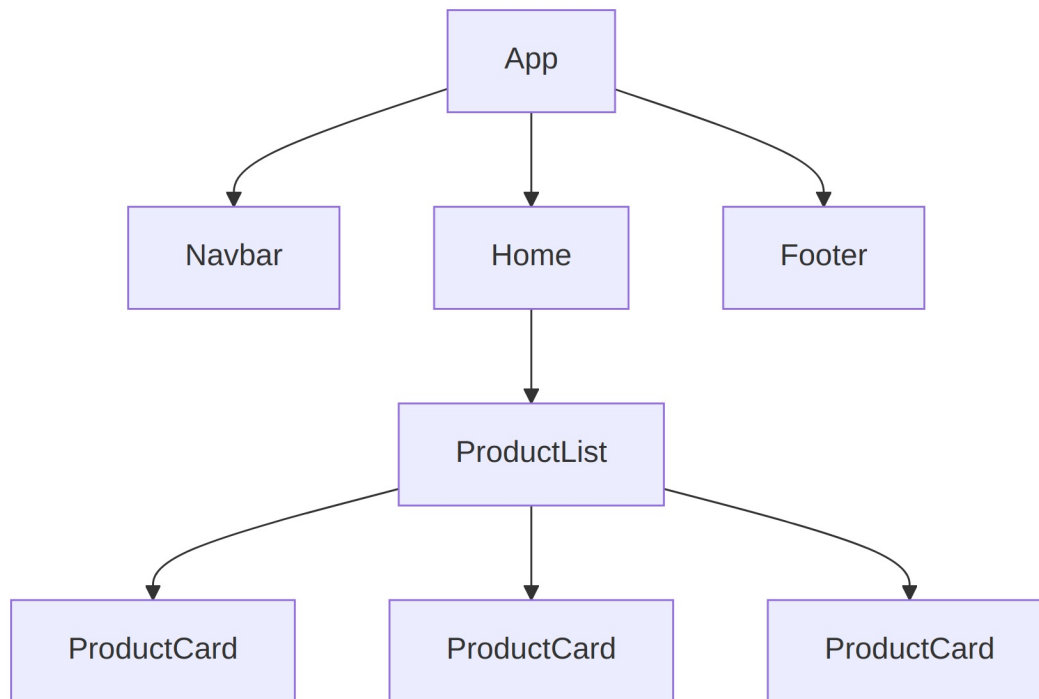
```
// Navbar.jsx — a component is just a function returning JSX
function Navbar() {
  return (
    <nav>
      <h1>MyShop</h1>
    </nav>
  );
}

export default Navbar;
```

Key JSX rules: one root element (or `<>...</>` fragment), `className` not `class`, `{ }` to drop JS values into markup.

2. Component Architecture

2.1 Composition — build big UIs from small pieces



Component tree diagram

Each box is its own file, and each does one job (Single Responsibility — the same principle from folder-structure planning, now applied to the frontend).

2.2 Props — passing data down

```
// ProductCard.jsx
function ProductCard({ name, price, image }) {
  return (
    <div className="card">
      <img src={image} alt={name} />
      <h3>{name}</h3>
      <p>Rs. {price}</p>
    </div>
  );
}

export default ProductCard;
```

```
// ProductList.jsx
import ProductCard from "../ProductCard";

const products = [
  { id: 1, name: "Notebook", price: 60, image: "/notebook.jpg" },
  { id: 2, name: "Pen", price: 10, image: "/pen.jpg" },
];

function ProductList() {
  return (
    <div className="grid">
      {products.map((p) => (
        <ProductCard key={p.id} name={p.name} price={p.price} image={p.image} />
      ))}
    </div>
  );
}

export default ProductList;
```

Note the `key` prop above — it's how React tracks which list item is which across re-renders, and forgetting it is one of the most common warnings you'll run into.

2.3 The `children` prop (for wrapper/layout components)

```
function Card({ children }) {
  return <div className="card-wrapper">{children}</div>;
}

// Usage:
<Card>
  <h2>Any content goes here</h2>
</Card>
```

3. State Management

3.1 `useState` — a component's own memory

```
import { useState } from "react";

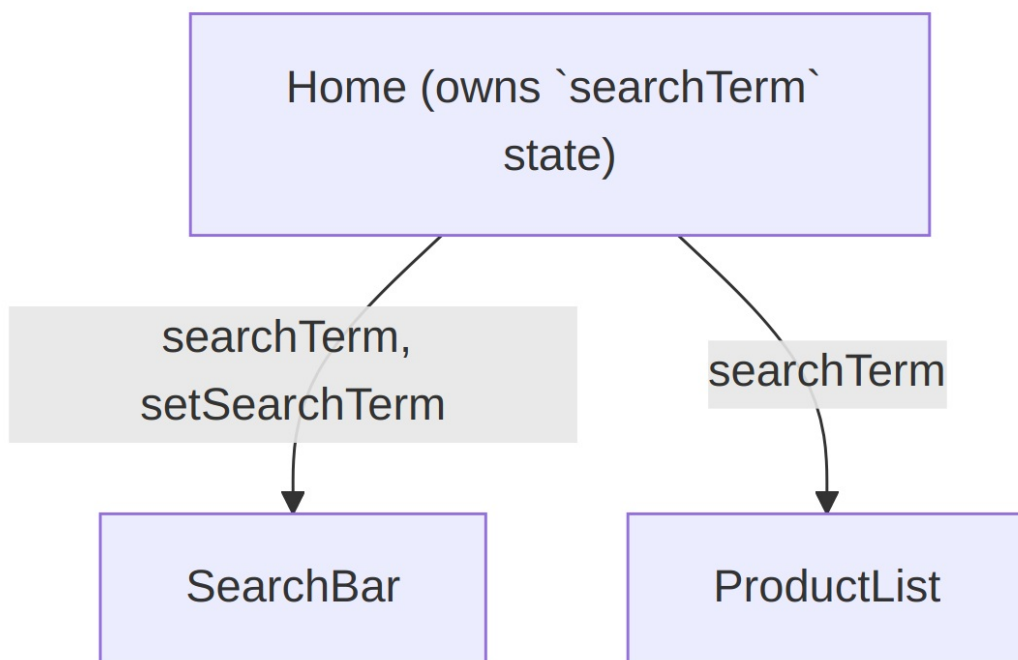
function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>+1</button>
    </div>
  );
}
```

A normal JS variable resets every re-render. `useState` gives you a variable React remembers between renders — and `setCount` is the only way to trigger a re-render with the new value.

3.2 Lifting state up

Problem: two sibling components need the *same* state (e.g., `SearchBar` and `ProductList` both need the search term). Solution: move the state to their common parent, pass it down as props.



Lifting state up diagram

```
function Home() {
  const [searchTerm, setSearchTerm] = useState("");

  return (
    <>
      <SearchBar value={searchTerm} onChange={setSearchTerm} />
      <ProductList filter={searchTerm} />
    </>
  );
}
```

3.3 Prop drilling → Context API

Lifting state up works for 2 levels. Once `Cart` state needs to reach `Navbar`, `ProductCard`, and a `CartPage` — all in different branches of the tree — passing props down through every layer (“prop drilling”) gets unmanageable. Context solves this by letting any component subscribe directly.

```
// context/CartContext.jsx
import { createContext, useContext, useState } from "react";

const CartContext = createContext();

export function CartProvider({ children }) {
  const [cartItems, setCartItems] = useState([]);

  const addToCart = (product) => setCartItems((prev) => [...prev, product]);

  return (
    <CartContext.Provider value={{ cartItems, addToCart }}>
      {children}
    </CartContext.Provider>
  );
}

// Custom hook for easy access anywhere in the tree
export function useCart() {
  return useContext(CartContext);
}
```

```
// Any deeply nested component:
import { useCart } from "../context/CartContext";

function ProductCard({ product }) {
  const { addToCart } = useCart();
  return <button onClick={() => addToCart(product)}>Add to Cart</button>;
}
```

Rule of thumb: Use `useState` for state one component owns and uses alone. Lift state up when 2–3 nearby components share it. Reach for Context when state needs to be read/written from many unrelated places in the tree (cart, logged-in user, theme).

4. Routing with React Router

4.1 Setup

```
npm install react-router-dom
```

```
// main.jsx
import { BrowserRouter } from "react-router-dom";
import App from "./App";

createRoot(document.getElementById("root")).render(
  <BrowserRouter>
    <App />
  </BrowserRouter>
);
```

4.2 Defining routes

```
// App.jsx
import { Routes, Route } from "react-router-dom";
import Home from "./pages/Home";
import ProductDetails from "./pages/ProductDetails";
import Cart from "./pages/Cart";
import NotFound from "./pages/NotFound";

function App() {
  return (
    <Routes>
      <Route path="/" element={<Home />} />
      <Route path="/products/:id" element={<ProductDetails />} />
      <Route path="/cart" element={<Cart />} />
      <Route path="*" element={<NotFound />} />
    </Routes>
  );
}
```

4.3 Navigating — `Link`, not `<a>`

```
import { Link } from "react-router-dom";

<Link to="/cart">View Cart</Link>
```

Why not `<a href="/cart">`? A plain `<a>` triggers a full page reload from the server; `Link` does client-side navigation instantly, keeping all your React state intact (no server round-trip needed for a page that's already loaded in the browser).

4.4 Dynamic routes — `useParams`

```
// pages/ProductDetails.jsx
import { useParams } from "react-router-dom";

function ProductDetails() {
  const { id } = useParams(); // from /products/:id
  // later: fetch product data using this id
  return <h2>Showing details for product {id}</h2>;
}
```

4.5 Programmatic navigation — `useNavigate`

```
import { useNavigate } from "react-router-dom";

function CheckoutButton() {
  const navigate = useNavigate();
  const handleCheckout = () => {
    // after some logic...
    navigate("/order-success");
  };
  return <button onClick={handleCheckout}>Checkout</button>;
}
```

4.6 Nested routes & shared layout (`<Outlet />`)

```
<Routes>
  <Route path="/" element={<MainLayout />}>
    <Route index element={<Home />} />
    <Route path="products/:id" element={<ProductDetails />} />
  </Route>
</Routes>
```

```
// MainLayout.jsx
import { Outlet } from "react-router-dom";
import Navbar from "../components/Navbar";

function MainLayout() {
  return (
    <>
      <Navbar />
      <main><Outlet /></main>
    </>
  );
}
```

This is how you avoid repeating `<Navbar />` in every single page component — it's the standard pattern, not an advanced trick.



Routing flow diagram

5. Putting It Together — Mini Shopping App Structure

```
src/
├─ context/CartContext.jsx
├─ components/
│   └─ Navbar.jsx
│   └─ ProductCard.jsx
├─ layouts/MainLayout.jsx
├─ pages/
│   └─ Home.jsx
│   └─ ProductDetails.jsx
│   └─ Cart.jsx
├─ App.jsx
└─ main.jsx
```

Data flow: `main.jsx` wraps `<App/>` in `<BrowserRouter>` and (once state is needed globally) `<CartProvider>` → `App.jsx` defines routes under `MainLayout` → `Navbar` uses `useCart()` to show item count → `ProductCard` uses `useCart()` to add items → `useNavigate` sends the user to `/cart` after checkout.

6. Hands-On Lab (40 min, pairs)

Building on your capstone idea:

1. Create at least 3 page-level components and wire them up with React Router (`/` , a dynamic detail route, and one more)
 2. Build one shared layout with a `Navbar` using `<Outlet />`
 3. Create a Context (e.g., `FavoritesContext` or your capstone's equivalent) and use it from at least 2 unrelated components
 4. Use `useParams` in your detail page and `useNavigate` on at least one button
 5. Push this as a `feature/routing-and-state` branch and open a PR
-

7. Quick Assessment

1. What's the difference between props and state?
 2. Why does React ask for a `key` prop on list items?
 3. When should you lift state up vs. reach for Context?
 4. Why is `<Link to="/cart">` preferred over `<a href="/cart">` inside a React app?
 5. What does `<Outlet />` do inside a layout route?
 6. What hook would you use to read `:id` from `/products/:id`?
-

8. Common Mistakes to Avoid

- Mutating state directly (`cartItems.push(item)`) instead of `setCartItems([...cartItems, item])` — React won't re-render
 - Forgetting `key` on mapped lists, or using array index as key when the list can reorder
 - Wrapping the whole app in Context "just in case" instead of scoping providers to where they're actually needed
 - Using `<a>` for internal navigation, causing a full page reload and losing all app state
-

Notes prepared for Full Stack Learning — MERN course, Unit 2.